

ATTIVITÀ PROGETTUALE - SICUREZZA DELL'INFORMAZIONE

A.R.E.S.

Automated Reconnaissance & Enumeration System



Candidato: Stefano Casini

Tutor: Vincenzo Mucciante (Cryptonet Labs s.r.l.)

Alma Mater Studiorum - Bologna

Limiti Approccio Manuale



Eterogeneità

Output frammentati (XML, JSON, Grepable) che richiedono parsing manuali e onerosi per ogni singolo tool.



Incoerenza

Difficoltà nel correlare servizi e vulnerabilità esposte sullo stesso IP quando provengono da sensori diversi.



Artifact Chaos

Gestione disordinata dei file raw, difficili da catalogare e associare univocamente a target o timestamp.



Obiettivi del Progetto

- **Orchestrazione Ibrida:** Integrazione di scansioni attive (Nmap/Masscan) e passive (Smap).
- **Normalizzazione:** Traduzione di formati proprietari in un modello dati DTO unificato.
- **Astrazione Operativa:** Configurazione dichiarativa tramite YAML per mascherare la complessità.
- **Interoperabilità:** Output JSON pronto per pipeline SIEM o Vulnerability Scanners.

Focus: Automazione

Ridurre drasticamente i tempi di Reconnaissance aumentando la precisione dei dati raccolti rispetto ai test manuali.



Active vs Passive

Il Trade-off della Visibilità

Ogni attività è disciplinata dalle **Rules of Engagement (ROE)**, che definiscono il perimetro legale e l'intrusività consentita.

Active

Interazione Diretta

Utilizzo di strumenti come **Nmap** e **Masscan**.

Offre la massima precisione dei dati ma comporta un elevato rischio di rilevamento da parte di sistemi **IDS/IPS**.

Passive

Zero-touch

Integrazione con **Smap** e **Shodan**.

Garantisce totale invisibilità poiché i dati sono basati esclusivamente su **record storici** esistenti.



Architettura del Sistema



CONFIG

Parsing YAML e task generation.



LAUNCH

Esecuzione Multi-thread.



PARSING

Normalizzazione in DTO.



MERGE

Data Fusion & Accumulo Evidenze.



REPORT

Generazione JSON finale.



Pattern & Estensibilità

Strategy Pattern

Utilizzo di classi astratte (CommandGenerator e BaseParser) per permettere l'aggiunta di nuovi tool senza modificare il core.

Data Transfer Objects

Modello dati fortemente tipizzato per Host, Port e OS.
Garantisce un contratto strutturale tra parser e merger.

- › Disaccoppiamento totale Launcher/Parser.
- › Supporto socket raw (sudo) per Masscan.
- › Gestione asincrona dei risultati.
- › Normalizzazione XML/JSON/Raw.



Modulo Merger & Data Fusion

L'algoritmo risolve attivamente le sovrapposizioni tra tool diversi, applicando una logica di accumulo.

Target IP	Porta	Fonte Nmap	Fonte Masscan	Stato Finale A.R.E.S.
192.168.1.10	80/tcp	Open (HTTP)	Open (Banner)	Aggregato
192.168.1.15	22/tcp	-	Open	Discovery Rapido
192.168.1.10	443/tcp	Open (v.1.2)	-	Deep Scan



Configurazione YAML

Pannello di Controllo

La logica non è hard-coded: il file config.yml governa l'intero processo.

- > Definizione dei target e range di porte.
- > Configurazione dei thread paralleli.
- > Flag specifici per ogni tool (es. rate-limit).
- > Importazione di scansioni storiche.

```
# =====  
# ARES CONFIGURATION FILE  
# =====  
  
output_report_path: "output"  
# -----  
# SCAN MODE  
# -----  
mode:  
  enable_scan: true          # enable or disable the scanning phase  
  n_threads: 3 # to choose the number of parallel scans  
  base_output_raw_path: "output/raw" # base path to store raw scan outputs  
  
# NMAP  
launcher_nmap:  
  enabled: true              # enable or disable Nmap module  
  top_port_range: 100  
  modes:  
    .  
    .  
    .  
  
# MASSCAN  
launcher_masscan:  
  enabled: true              # enable or disable Masscan module  
  top_ports: 100  
  rate: 1000  
  modes:  
    .  
    .  
    .  
  
# SMAP  
launcher_smap:  
  enabled: true              # enable or disable Nmap module  
  top_port_range: 100  
  modes:  
    .  
    .  
    .  
  
# -----  
# IMPORT FILE  
# -----  
import_files:  
  - "output/raw/<timestamp>_<target>/nmap_top_tcp_ports.xml" how to import example
```



Analisi del Report Finale

JSON Output Preview

```
{
  "total_hosts": 12,
  "commands": [
    "sudo masscan --top-ports 100 --rate 5000 ...",
    "nmap -sV -p 80,443 ...",
    "smmap -p1-65535 ..."
  ],
  "hosts": {
    "10.0.0.15": {
      "ip": "10.0.0.15",
      "discovery_commands": ["masscan", "smmap"],
      "ports": {
        "80/tcp": [
          {
            "state": "open",
            "service": "http",
            "banner": "Apache/2.4.41 (Ubuntu)",
            "source": "nmap",
            "reason": "syn-ack"
          },
          {
            "state": "open",
            "source": "masscan",
            "ttl": 56
          }
        ],
        "443/tcp": [
          {
            "state": "open",
            "source": "smmap",
            "reason": "shodan-api"
          }
        ]
      },
      "hostnames": {
        "web-prod-01.local": ["nmap"]
      },
      "os": {
        "name": "Linux 5.x",
        "accuracy": 98
      },
      "cpes": ["cpe:/o:linux:linux_kernel:5"]
    },
    "10.0.0.18": {
      "...": "Ulteriori host aggregati qui"
    }
  }
}
```

Accumulo delle Evidenze

Nessun dato viene sovrascritto. Il JSON mantiene la traccia di ogni sensore che ha rilevato la porta.



Metadati Globali

Audit trail dei comandi eseguiti.



Correlazione Fonti

Confronto tra stato attivo (Nmap) e storico (Shodan).



Machine Readable

Pronto per l'ingestione in pipeline esterne.



Sviluppi futuri



Web GUI Integrata

Sviluppo di un'interfaccia utente moderna per la gestione semplificata delle scansioni.



Integrazione Nuclei

Implementazione del motore Nuclei per scansioni di vulnerabilità basate su template.



Containerizzazione

Dockerizzazione dell'intero ecosistema A.R.E.S. per un deployment rapido e isolato.



Grazie per l'attenzione



Contatti

stefano.casini2@studio.unibo.it



Repository

github.com/StefanoCasini/ARES



ARES